

DEVOPS & SOFTWARE ENGINEERING

Trunk-Based Development

Accelerating Software Delivery through Continuous Integration and Short-Lived Branches

Presented by –
Prabhjot Singh
Pavela Parvez
Sadia Lima

Sadia Lima
Pavela Parvez

What is Trunk-Based Development?

Mainline Integration

Trunk-Based Development (TBD) is a branching model where all developers integrate code changes into a single central branch—the "**trunk**" or "**main**"—multiple times a day.

It replaces isolated, long-lived feature branches with small, frequent merges to maintain continuous deployment readiness.

The Modern CI Enabler

TBD is a foundational requirement for true **Continuous Integration (CI)** and **Continuous Delivery (CD)**.

By keeping commits small, development teams minimize friction, avoid "merge hell," and establish a high-velocity feedback loop across the engineering organization.

The Three Pillars of TBD



Short-Lived Branches

Feature branches exist for hours or a day at most. Small code increments mean review and integration happen almost instantaneously.



Automated Testing

Every commit triggers automated build and test pipelines. The trunk must remain "green" and deployable to production at all times.

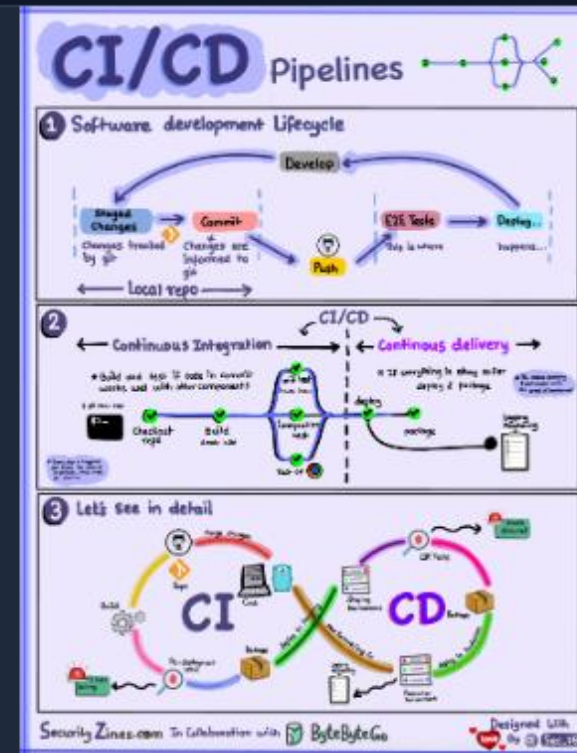


Feature Toggles

Unfinished features are wrapped in feature flags (toggles) so incomplete code can safely merge into production without exposure to end users.

The Daily Developer Workflow

- **Pull & Sync Frequently:** Developers start the day by pulling the latest changes from main to stay synchronized.
- **Small Batch Changes:** Work is broken down into small, digestible commits (typically < 200 lines of code).
- **Rapid Pull Requests:** Code reviews happen quickly on short branches, taking minutes rather than days.
- **Automated CI Verification:** Automated pipelines validate builds and run test suites before merging back to trunk.



Trunk-Based Development vs GitFlow

Comparison Metric	Trunk-Based Development	GitFlow Model
Branch Lifetime	Short-lived (Hours to 1 day)	Long-lived (Weeks to months)
Branch Complexity	Single trunk + brief feature branches	Main, Develop, Feature, Release, Hotfix
Merge Friction	Low (frequent, small changes)	High (infrequent, massive merges)
Feedback Loop	Immediate (automated per commit)	Delayed until late integration phase
Deployment Cadence	Continuous / Daily releases	Scheduled milestone releases

Key Advantages (Pros)

-  **Eliminates Merge Hell:** Small, daily merges prevent complex code drift and painful, multi-day merge conflict resolutions.
-  **Faster Time-to-Market:** Features and bug fixes reach production in continuous, incremental iterations rather than delayed batches.
-  **High Visibility & Transparency:** Team members gain real-time visibility into what peers are building, fostering stronger collaboration.
-  **Early Bug Detection:** Automated testing on every commit catches defects immediately when they are easiest and cheapest to fix.

Challenges & Disadvantages (Cons)



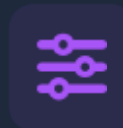
High CI/CD Dependency

Requires mature automated testing and fast CI pipelines. If CI tests are slow or flaky, the whole development flow stalls.



Cultural Shift & Discipline

Developers must learn to break large tasks into atomic commits and maintain rigorous discipline around trunk stability.



Flag Management Overhead

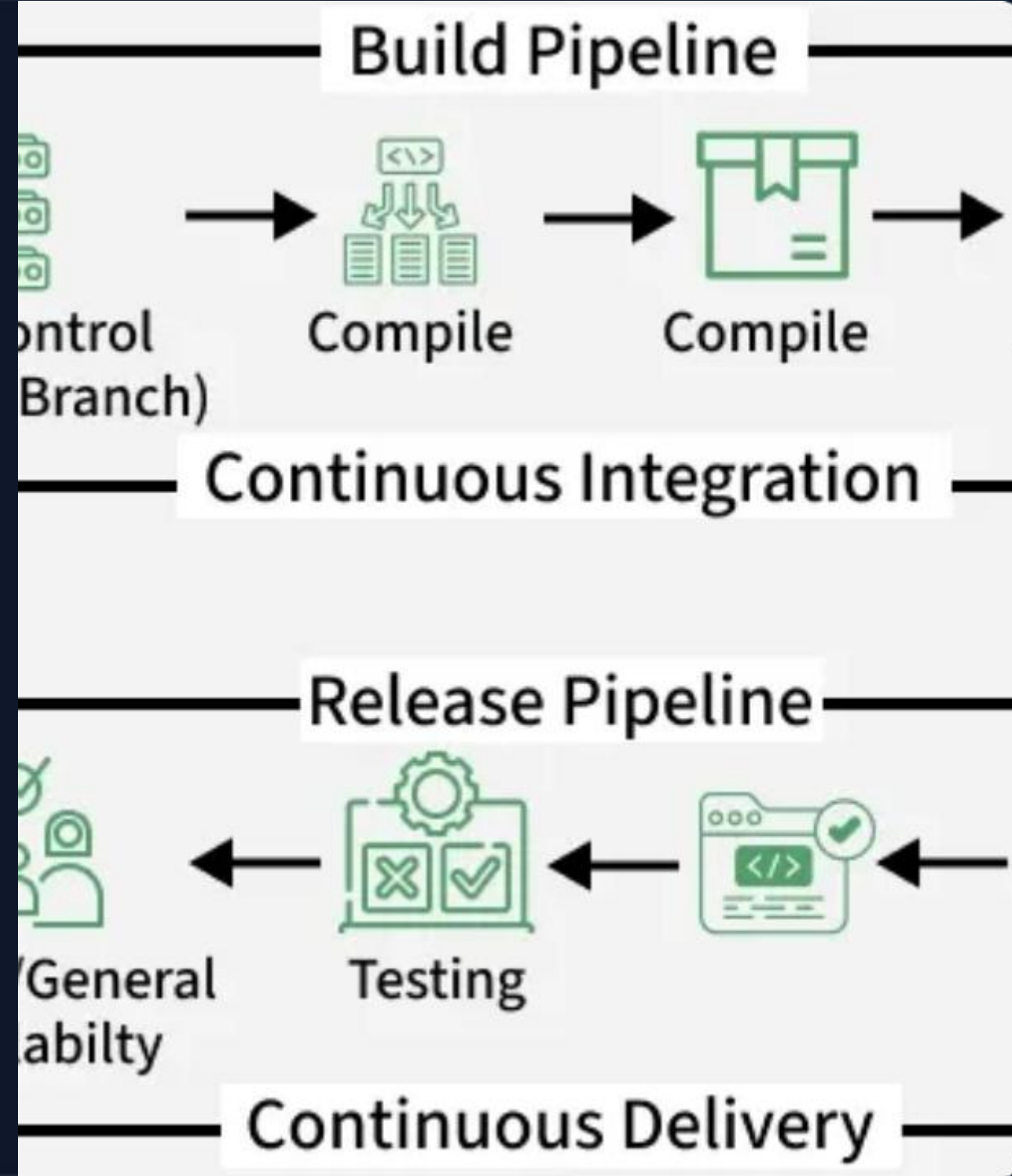
Heavy reliance on feature flags requires strict governance to clean up obsolete flags and avoid tech-debt accumulation.

Essential Technical Enablers

Continuous Integration & Feature Flags

To succeed with Trunk-Based Development, organizations must invest in two primary technical foundations:

- 1. Robust Automated Testing:** Unit, integration, and security checks must run in under 10 minutes to maintain fast feedback.
- 2. Feature Flagging Architecture:** Decouples feature deployment from feature release, empowering developers to push dark code safely to production.



Quantifiable Impact on Engineering Metrics

Deployment Frequency

+400% (Multiple / Day)

Lead Time for Changes

-75% (Hours vs Weeks)

Time Spent Merging

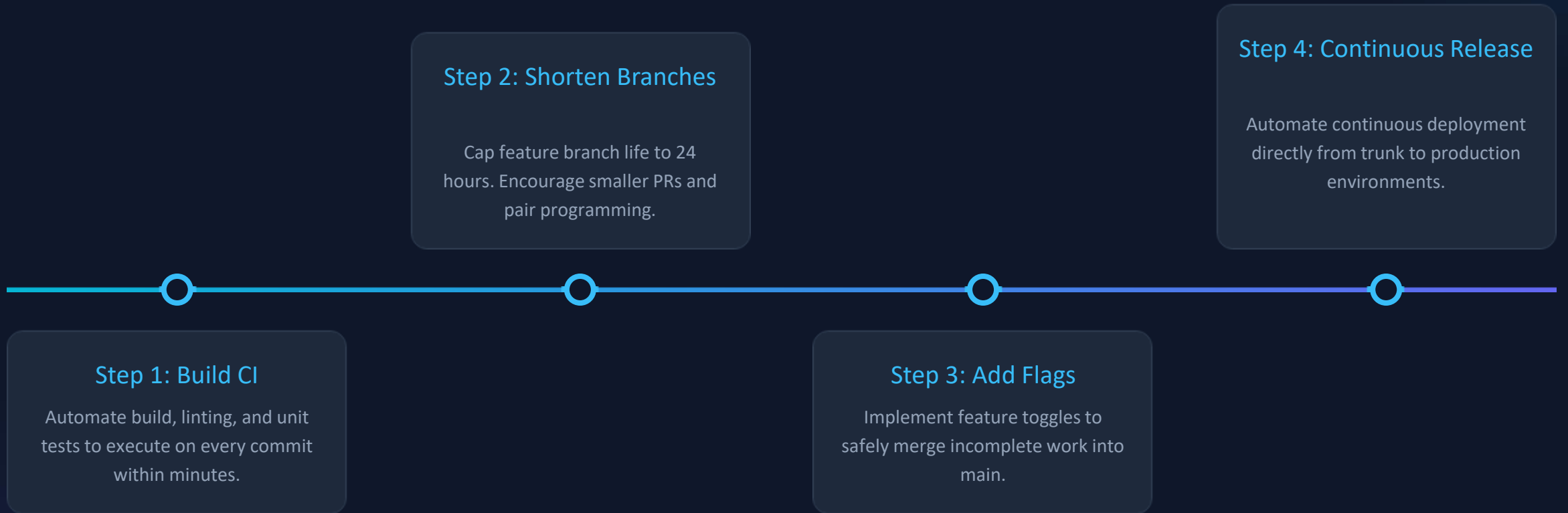
-80% Reduction

Change Failure Rate

-50% Reduction

DORA research confirms teams using Trunk-Based Development achieve Elite DevOps status with significantly faster recovery times and higher release stability.

4-Step Strategy to Adopt TBD



Best Practices for Long-Term Success

-  **Keep Changes Granular:** Aim for minor, single-purpose commits that are easy to review and revert if needed.
-  **Prioritize Fast Build Times:** Keep the core CI pipeline running under 5–10 minutes to maintain momentum.
-  **Prompt Code Reviews:** Treat incoming Pull Requests as urgent to prevent branches from aging.
-  **Clean Up Feature Flags:** Schedule regular technical debt cleanup sprints to remove obsolete toggles.

Image Sources



<https://assets.bytebytego.com/diagrams/0140-ci-cd-pipeline.png>

Source: bytebytego.com



<https://media.geeksforgeeks.org/wp-content/uploads/20250821153350614262/cicd12.webp>

Source: www.geeksforgeeks.org

Thank you